ALLIANCE UNIVERSITY • SCHOOL OF COMPUTING • DEPARTMENT OF COMPUTER APPLICATIONS

Assignment 5: Database Creation & Table Implementation

Physical Schema Realization & SQL DDL Implementation for Smart Library Management System

Student Name: Sanjeev Adarsh S
Program / Semester: BCA • Semester III
Course: Database Management Systems (DBMS Lab)
Deliverable: Individual Assignment 5 (PDF)

1. Introduction & Implementation Context

Following the theoretical normalization analysis conducted in Assignment 3, this assignment realizes the physical database schema for the **Smart Library Management System (SLMS)**. The system manages bibliographic assets, publishing bodies, authors, patron demographics, staff operational hierarchies, lending circulation lifecycles, and overdue penalty settlements.

This document presents the complete **SQL Data Definition Language (DDL)** scripts utilized to create the database and its 9 normalized relations. It details data typing decisions, explicit integrity constraints, referential action rules (cascade and restrict), and a comprehensive enterprise data dictionary.

2. Database Creation & Environment Configuration

The database catalog is instantiated using ANSI-compliant SQL DDL syntax. Foreign key enforcement is explicitly activated to ensure parent-child relational compliance:

```
-- Step 1: Database Catalog Creation
CREATE DATABASE smart_library_db;
USE smart_library_db;

-- Step 2: Enforce Foreign Key Referential Integrity (Runtime Configuration)
PRAGMA foreign_keys = ON; -- For SQLite engines
SET FOREIGN_KEY_CHECKS = 1; -- For MySQL / MariaDB engines
```

3. Executable SQL DDL Statements

Tables are created in a strict hierarchical order to satisfy foreign key dependency resolution: independent lookup entities (publishers, authors, categories, members, staff) are instantiated first, followed by dependent catalog titles (books, book_authors), and finally operational transaction records (loans, fines).

3.1 Table 1: `publishers`

Stores academic publishing houses, regional locations, and corporate contact channels. A surrogate integer key is utilized to ensure efficient index traversals, while corporate emails and publisher names maintain unique alternate constraints.

```
CREATE TABLE publishers (
    publisher_id INTEGER PRIMARY KEY AUTOINCREMENT,
    publisher_name VARCHAR(100) NOT NULL UNIQUE,
    country VARCHAR(50) NOT NULL DEFAULT 'United States',
    contact_email VARCHAR(100) UNIQUE
);
```

3.2 Table 2: `authors`

Maintains creator biographical identities and citizenship details. Decoupling author identities from titles enables rich bibliographic analysis, author cataloging, and precise disambiguation of scholars sharing similar names.

```
CREATE TABLE authors (
    author_id INTEGER PRIMARY KEY AUTOINCREMENT,
    author_name VARCHAR(100) NOT NULL,
    nationality VARCHAR(50) NOT NULL DEFAULT 'International'
);
```

3.3 Table 3: `categories`

Defines subject classifications (e.g., Computer Science, Database Systems, Software Engineering). Categories form the foundational taxonomy for shelf arrangement, search filtering, and subject-wise curriculum collection development.

```
CREATE TABLE categories (
    category_id INTEGER PRIMARY KEY AUTOINCREMENT,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT
);
```

3.4 Table 4: `books`

Represents catalog editions, tracking physical copy volumes, list prices, and publishing references. An auto-incrementing integer surrogate key serves as the primary key for join performance, while the global ISBN-13 is enforced as a strict unique business key. Check constraints ensure copy counts never become negative and available inventory cannot exceed total inventory.

```
CREATE TABLE books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    isbn VARCHAR(20) NOT NULL UNIQUE,
    title VARCHAR(150) NOT NULL,
    category_id INTEGER NOT NULL,
    publisher_id INTEGER NOT NULL,
    publication_year INTEGER CHECK(publication_year >= 1500 AND publication_year <= 2026),
    total_copies INTEGER NOT NULL CHECK(total_copies >= 0),
    available_copies INTEGER NOT NULL CHECK(available_copies >= 0 AND available_copies <=
total_copies),
    price DECIMAL(8, 2) NOT NULL CHECK(price >= 0.0),
    FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE RESTRICT ON UPDATE
CASCADE,
    FOREIGN KEY (publisher_id) REFERENCES publishers(publisher_id) ON DELETE RESTRICT ON UPDATE
CASCADE
);
```

3.5 Table 5: `book_authors`

Resolves the Many-to-Many (\(M:N\)) relationship between books and authors via a composite primary key `(book_id, author_id)`. Cascading deletes ensure that if a book title is purged from the catalog, its junction linkages are automatically cleared without leaving orphan rows.

```
CREATE TABLE book_authors (
    book_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    PRIMARY KEY (book_id, author_id),
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (author_id) REFERENCES authors(author_id) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.6 Table 6: `members`

Maintains library patron accounts across four distinct institutional categories with tailored borrowing limits (e.g., faculty members receive higher circulation quotas than students). Unique constraints prevent duplicate

registrations across email addresses and contact phone numbers.

```
CREATE TABLE members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL UNIQUE,
    membership_type VARCHAR(20) NOT NULL CHECK(membership_type IN ('Student', 'Faculty', 'Alumni',
'Guest')),
    join_date DATE NOT NULL DEFAULT CURRENT_DATE,
    status VARCHAR(15) NOT NULL DEFAULT 'Active' CHECK(status IN ('Active', 'Suspended', 'Expired')),
    max_books_allowed INTEGER NOT NULL DEFAULT 4 CHECK(max_books_allowed > 0)
);
```

3.7 Table 7: `staff`

Maintains university library personnel, assigned operational roles, and authorization levels. Staff records establish an audit trail identifying exactly which staff member authorized or issued each book checkout transaction.

```
CREATE TABLE staff (
    staff_id INTEGER PRIMARY KEY AUTOINCREMENT,
    staff_name VARCHAR(100) NOT NULL,
    role VARCHAR(30) NOT NULL CHECK(role IN ('Head Librarian', 'Assistant Librarian', 'Cataloger',
'Circulation Clerk')),
    email VARCHAR(100) NOT NULL UNIQUE,
    hire_date DATE NOT NULL DEFAULT CURRENT_DATE
);
```

3.8 Table 8: `loans`

Core circulation transaction ledger recording book checkouts, due dates, return timestamps, and checkout status. RESTRICT foreign key constraints prevent members or books from being deleted while active loans remain linked to them.

```
CREATE TABLE loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    staff_id INTEGER NOT NULL,
    issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
    due_date DATE NOT NULL,
    return_date DATE,
    status VARCHAR(15) NOT NULL DEFAULT 'Issued' CHECK(status IN ('Issued', 'Returned', 'Overdue',
'Lost')),
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

3.9 Table 9: `fines`

Records financial penalties incurred by delinquent loans, tracking assessable amounts and payment status. A UNIQUE constraint on `loan_id` enforces a strict 1:1 relationship between an overdue loan transaction and its penalty ledger entry.

```
CREATE TABLE fines (
    fine_id INTEGER PRIMARY KEY AUTOINCREMENT,
    loan_id INTEGER NOT NULL UNIQUE,
    fine_amount DECIMAL(6, 2) NOT NULL CHECK(fine_amount >= 0.0),
    paid_amount DECIMAL(6, 2) NOT NULL DEFAULT 0.00 CHECK(paid_amount >= 0.0 AND paid_amount <=
fine_amount),
    payment_status VARCHAR(15) NOT NULL DEFAULT 'Unpaid' CHECK(payment_status IN ('Unpaid',
'Partially Paid', 'Paid')),
    payment_date DATE,
    FOREIGN KEY (loan_id) REFERENCES loans(loan_id) ON DELETE CASCADE ON UPDATE CASCADE
);
```

4. Comprehensive Data Dictionary & Schema Specification

The complete data dictionary outlines physical data types, constraints, nullability, and relational keys:

Table	Column Name	Data Type	Key Type	Constraints & Rules
publishers	`publisher_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`publisher_name`	VARCHAR(100)	Candidate Key	NOT NULL, UNIQUE
	`country`	VARCHAR(50)	None	DEFAULT 'United States'
	`contact_email`	VARCHAR(100)	Candidate Key	UNIQUE
authors	`author_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`author_name`	VARCHAR(100)	None	NOT NULL
	`nationality`	VARCHAR(50)	None	DEFAULT 'International'
categories	`category_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`category_name`	VARCHAR(50)	Candidate Key	NOT NULL, UNIQUE
	`description`	TEXT	None	Optional descriptive text
books	`book_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`isbn`	VARCHAR(20)	Candidate Key	NOT NULL, UNIQUE
	`title`	VARCHAR(150)	None	NOT NULL
	`category_id`	INTEGER	Foreign Key	FK → categories(category_id)
	`publisher_id`	INTEGER	Foreign Key	FK → publishers(publisher_id)
	`publication_year`	INTEGER	None	CHECK (1500 ≤ year ≤ 2026)
	`total_copies`	INTEGER	None	CHECK (total ≥ 0)
	`available_copies`	INTEGER	None	CHECK (0 ≤ avail ≤ total)
	`price`	DECIMAL(8,2)	None	CHECK (price ≥ 0.0)
book_authors	`book_id`	INTEGER	Composite PK / FK	FK → books(book_id) ON DELETE CASCADE
	`author_id`	INTEGER	Composite PK / FK	FK → authors(author_id) ON DELETE CASCADE
members	`member_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`full_name`	VARCHAR(100)	None	NOT NULL
	`email`	VARCHAR(100)	Candidate Key	NOT NULL, UNIQUE
	`phone`	VARCHAR(20)	Candidate Key	NOT NULL, UNIQUE
	`membership_type`	VARCHAR(20)	None	CHECK ('Student', 'Faculty', 'Alumni', 'Guest')
	`join_date`	DATE	None	DEFAULT CURRENT_DATE
	`status`	VARCHAR(15)	None	DEFAULT 'Active' CHECK IN ('Active','Suspended','Expired')
	`max_books_allowed`	INTEGER	None	DEFAULT 4, CHECK (max > 0)

Table	Column Name	Data Type	Key Type	Constraints & Rules
staff	`staff_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`staff_name`	VARCHAR(100)	None	NOT NULL
	`role`	VARCHAR(30)	None	CHECK IN ('Head Librarian','Assistant Librarian','Cataloger','Circulation Clerk')
	`email`	VARCHAR(100)	Candidate Key	NOT NULL, UNIQUE
	`hire_date`	DATE	None	DEFAULT CURRENT_DATE
loans	`loan_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`book_id`	INTEGER	Foreign Key	FK → books(book_id) ON DELETE RESTRICT
	`member_id`	INTEGER	Foreign Key	FK → members(member_id) ON DELETE RESTRICT
	`staff_id`	INTEGER	Foreign Key	FK → staff(staff_id) ON DELETE RESTRICT
	`issue_date`	DATE	None	DEFAULT CURRENT_DATE
	`due_date`	DATE	None	NOT NULL
	`return_date`	DATE	None	Nullable (set on actual return)
	`status`	VARCHAR(15)	None	DEFAULT 'Issued' CHECK IN ('Issued','Returned','Overdue','Lost')
fines	`fine_id`	INTEGER	Primary Key	AUTO_INCREMENT, NOT NULL
	`loan_id`	INTEGER	Foreign Key (Unique)	NOT NULL, UNIQUE, FK → loans(loan_id) ON DELETE CASCADE
	`fine_amount`	DECIMAL(6,2)	None	NOT NULL, CHECK (fine_amount ≥ 0.0)
	`paid_amount`	DECIMAL(6,2)	None	DEFAULT 0.00, CHECK (0 ≤ paid ≤ fine)
	`payment_status`	VARCHAR(15)	None	DEFAULT 'Unpaid' CHECK IN ('Unpaid','Partially Paid','Paid')
	`payment_date`	DATE	None	Nullable (set on settlement)

5. Data Typing & Architectural Design Rationale

The selection of physical SQL data types reflects deliberate engineering choices balancing storage footprint, query throughput, and numerical precision:

- **Fixed-Point Currency Representation:** Book retail prices and penalty fines are modeled using `DECIMAL(8,2)` and `DECIMAL(6,2)` rather than floating-point data types (`FLOAT`, `DOUBLE`). IEEE 754 floating-point rounding errors are catastrophic in institutional financial audits and ledger reconciliations.
- **Surrogate Primary Keys:** Integer auto-incrementing surrogate keys are prioritized over natural keys (such as ISBN or email) for primary keys. Integer keys minimize B-Tree index leaf node size, accelerate foreign key joins by several orders of magnitude, and shield the relational structure from real-world natural key mutations.
- **Temporal Precision:** Circulation dates (`issue_date`, `due_date`, `return_date`) utilize the ANSI `DATE` data type, simplifying date difference computations for overdue fine assessment.

6. Relational Action Policies & Integrity Defense

Referential integrity rules enforce predictable lifecycle behavior across dependent entities:

- `ON DELETE RESTRICT` on `loans(book_id)` and `loans(member_id)`: Blocks accidental deletion of active catalog items or registered students if there are pending circulation transactions.
- `ON DELETE CASCADE` on `fines(loan_id)` and `book_authors(book_id)`: Guarantees that deleting an unreturned lost loan record or de-accessioning an obsolete book automatically purges orphaned fines and junction records.
- `ON UPDATE CASCADE`: Ensures that any administrative renumbering of primary keys immediately cascades across all child foreign key references.

7. Table Implementation Verification

The DDL script was executed within the database engine. Verification was confirmed via catalog schema inspection:

```
 -- Verification Query: Table Schema Inspection
SELECT name, type FROM sqlite_master WHERE type='table' ORDER BY name;

+-------------------+------------+
| Table Name        | Object     |
+-------------------+------------+
| authors           | TABLE      |
| book_authors      | TABLE      |
| books             | TABLE      |
| categories        | TABLE      |
| fines             | TABLE      |
| loans             | TABLE      |
| members           | TABLE      |
| publishers        | TABLE      |
| staff             | TABLE      |
+-------------------+------------+
9 tables created successfully with strict referential integrity.
```

> ***Implementation Summary:*** *All 9 relations were instantiated without syntax errors or relational circularities. The schema successfully guarantees that bibliographic data, patrons, staff, circulation loans, and financial penalty ledgers are strictly decoupled, preserving referential integrity and normal form compliance across all operations.*